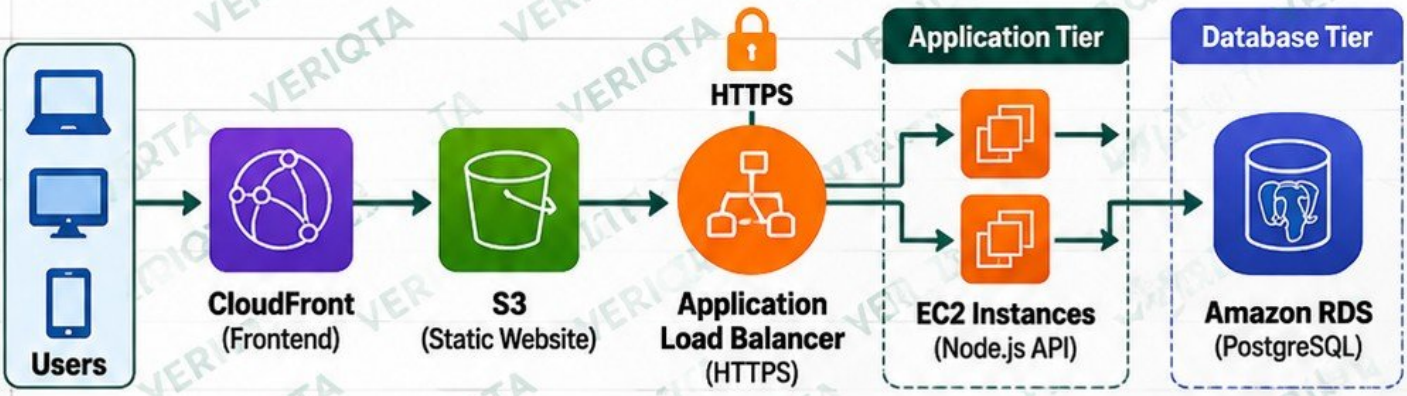
Build a Three-Tier App on AWS

The Project That Gets You Hired



Presentation Tier
React Frontend • S3 + CloudFront

Application Tier
Node.js API • ALB + EC2

Database Tier
Amazon RDS • PostgreSQL

EC2 · ALB · S3 · CloudFront · RDS · VPC

IAM · Systems Mamager · Auto Scaling · CloudWatch · Terraform · GitHub Actions

- REAL PROJECT
- HANDS-ON STEPS
- PRODUCTION BEST PRACTICES
- TROUBLESHOOTING
- PORTFOLIO READY
- INTERVIEW FOCUSED

The Three-Tier Project Brief

1. Project Overview

We will build **TaskFlow**, a simple task management web application on **AWS**. Users can create, view, update, and delete tasks. The application uses a three-tier architecture with a **React** frontend, a **Node.js** API backend, and a **PostgreSQL** database.

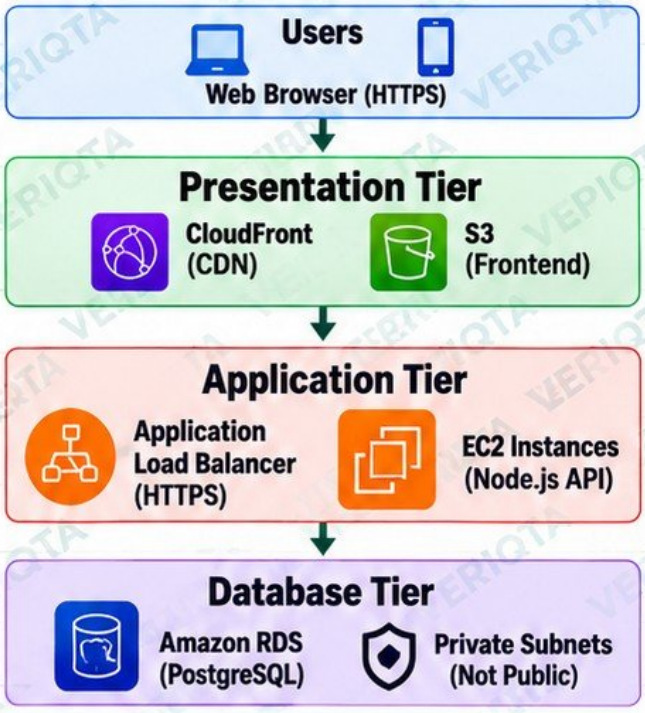
Goal

Build a real, production-style project using AWS services, follow best practices, and create a portfolio project you can confidently explain in a technical interview.

2. What We Will Build

1. **Create a task**
   Add a new task with a title and status.
2. **Read tasks**
   View all tasks or a single task.
3. **Update a task**
   Change the task title or status.
4. **Delete a task**
   Remove a task from the database.

3. High-Level Architecture



4. Users of the Application

- **Individuals** — Create and manage their own tasks.
- **Teams** — Collaborate on shared tasks.
- **Admins (Optional)** — Manage users and tasks.

5. Deliverables

- Complete source code (frontend and backend)
- AWS infrastructure (manual and Terraform)
- Deployed and working application
- Architecture diagram
- Deployment and testing steps
- Troubleshooting guide
- Project documentation (README)

6. Acceptance Criteria

- Users can create, read, update, and delete tasks.
- Application is accessible over **HTTPS**.
- Frontend is served via S3 + CloudFront.
- Backend runs on EC2 behind an ALB.
- Database is Amazon RDS (PostgreSQL) in private subnets.
- Follow AWS best practices for security and networking.
- Project is well documented and easy to deploy.

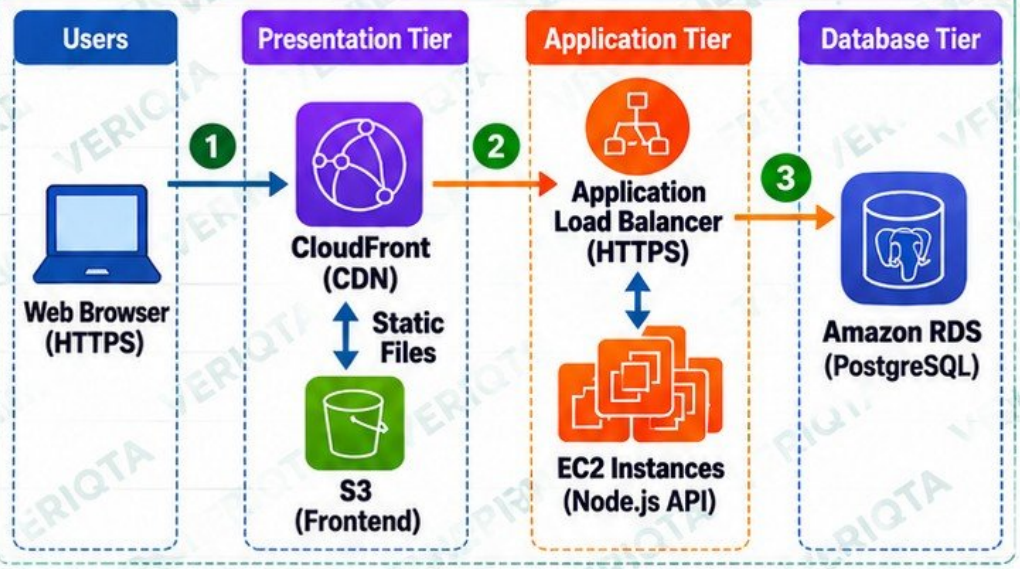
7. Why This Project Gets You Hired

- Uses real AWS services that companies use.
- Shows end-to-end DevOps and cloud engineering skills.
- Demonstrates understanding of production architecture.
- Includes security, networking, monitoring and best practices.
- Gives you a strong, portfolio-ready project.
- You can explain every component in interviews.

Understand the Architecture

A three-tier application separates the **frontend**, **backend**, and **database** into different layers. Each tier has a specific role, runs in the right place, and communicates securely.

1. Application Architecture



Why Separate?

- Better security
- Easier scaling
- Improved reliability
- Easier maintenance
- Real-world architecture
- Used in production

2. Request Flow (Step by Step)

1. User enters the application URL in a browser.
2. CloudFront serves the React frontend from S3 and routes API requests to the Application Load Balancer.
3. The Application Load Balancer forwards the request to a healthy EC2 instance running the Node.js API in private subnets.
4. The API processes the request and connects to Amazon RDS (PostgreSQL) in private subnets.
5. The response returns to the user following the same path.

3. Tiers Explained

Presentation Tier (Frontend)

- React application (static files)
- Stored in Amazon S3
- Delivered globally with CloudFront (CDN)
- Accessible over HTTPS

Application Tier (Backend)

- Node.js API (Express)
- Runs on EC2 instances in private subnets
- Placed behind an Application Load Balancer
- Handles business logic and API requests
- Communicates with RDS on private network

Database Tier (Database)

- Amazon RDS (PostgreSQL)
- Runs in private subnets
- Not publicly accessible
- Stores application data (tasks)
- Automated backups and encryption

4. Where Each Tier Runs in AWS

Tier	AWS Services	Location	Accessibility
Frontend	S3 + CloudFront	Global (Edge)	Public (HTTPS)
Backend	ALB + EC2	VPC (Private Subnets)	Private (via ALB)
Database	Amazon RDS	VPC (Private Subnets)	Private (No public access)

5. Key Takeaways

- Each tier has a specific role.
- Only the frontend and load balancer are public.
- The backend and database run in private subnets.
- This architecture is secure, scalable and reliable.
- It is the same pattern used in real companies.
- Understanding this flow helps you troubleshoot faster.

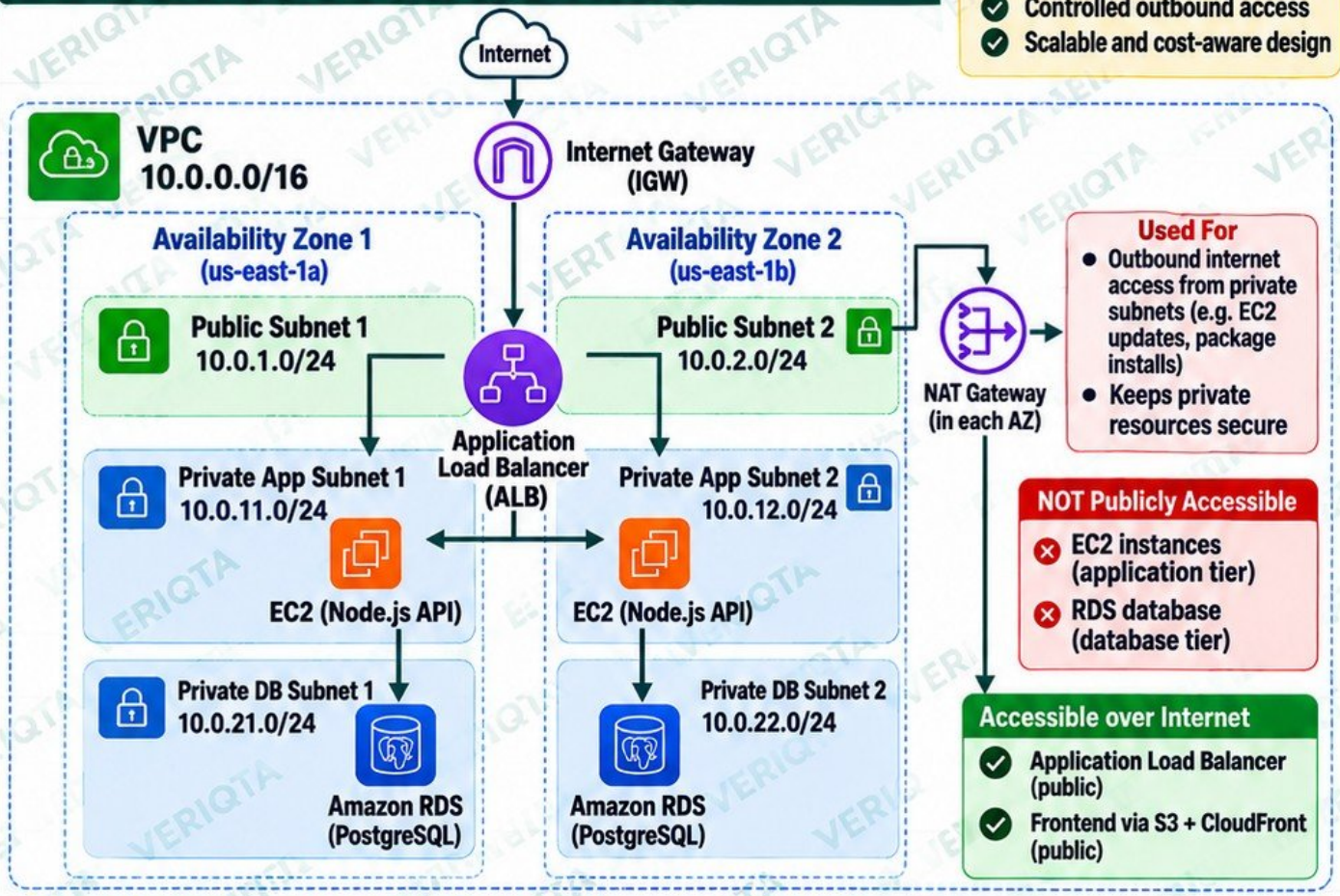
Design the AWS Network

We will design a secure VPC for our three-tier application with public and private subnets across two Availability Zones, route tables, internet access, and controlled outbound connectivity.

Design Goals

- High availability (2 AZs)
- Secure and isolated tiers
- Internet access for frontend
- Private backend and database
- Controlled outbound access
- Scalable and cost-aware design

1. VPC Architecture for Our Three-Tier App



2. Subnet Plan

Subnet Type	Availability Zone	CIDR Block	Purpose	Route Table
Public Subnet 1	us-east-1a	10.0.1.0/24	ALB, NAT Gateway	Public Route Table
Public Subnet 2	us-east-1b	10.0.2.0/24	ALB, NAT Gateway	Public Route Table
Private App Subnet 1	us-east-1a	10.0.11.0/24	EC2 (Node.js API)	Private Route Table
Private App Subnet 2	us-east-1b	10.0.12.0/24	EC2 (Node.js API)	Private Route Table
Private DB Subnet 1	us-east-1a	10.0.21.0/24	Amazon RDS (PostgreSQL)	Private Route Table
Private DB Subnet 2	us-east-1b	10.0.22.0/24	Amazon RDS (PostgreSQL)	Private Route Table

3. Route Table Design

Public Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Internet Gateway (IGW)

Private Route Table

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	NAT Gateway

4. Key Considerations

- Use two Availability Zones for high availability.
- Keep application and database subnets private.
- Use a NAT Gateway for controlled outbound access.
- Use security groups to control traffic between tiers.
- Plan CIDR blocks to allow future scaling.
- Use VPC endpoints for S3 (optional) to reduce NAT costs.

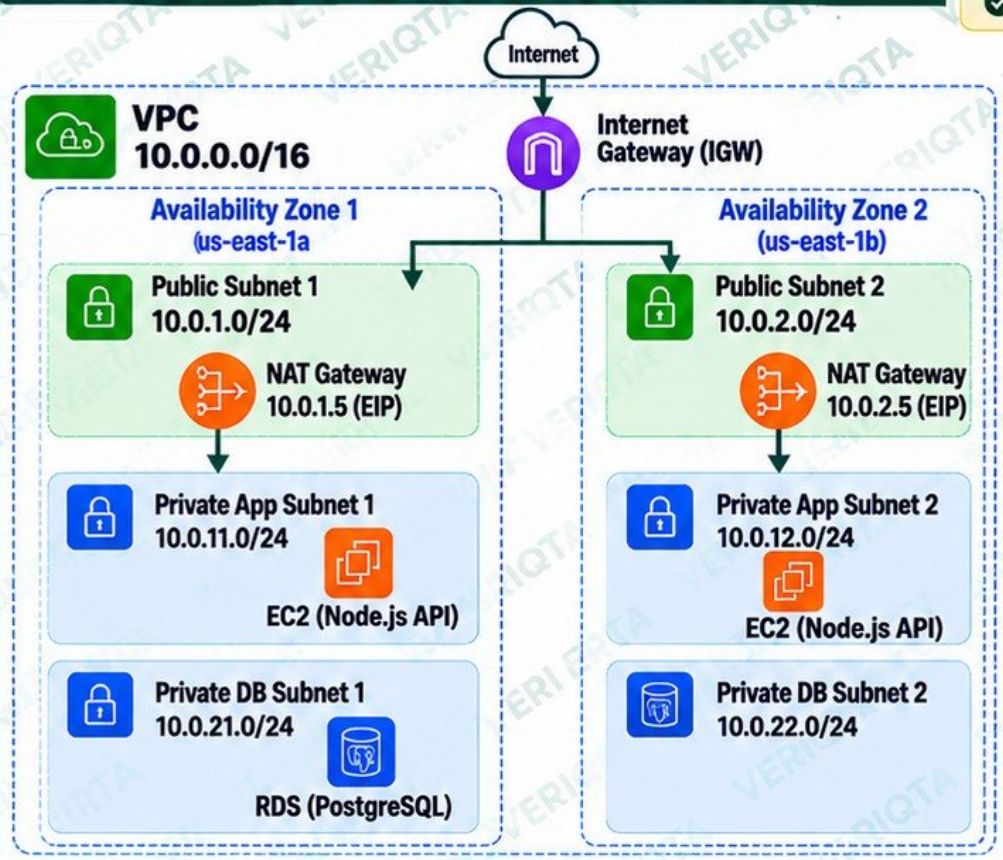
Build the VPC Foundation

In this step, we create the **VPC, subnets, internet gateway, route tables,** and **NAT gateway (or VPC endpoints).** We verify routing to make sure our network is working correctly before launching any application servers.

What We Will Build

- VPC with public and private subnets
- Internet gateway (IGW)
- NAT gateway (for private outbound)
- Route tables and routes
- Verify internet and AWS access

1. VPC Network Architecture (Target Design)



Key Components

- VPC (10.0.0.0/16)
- 2 Public Subnets (1 in each AZ)
- 2 Private App Subnets (1 in each AZ)
- 2 Private DB Subnets (1 in each AZ)
- Internet Gateway (IGW)
- NAT Gateway (1 per AZ) for outbound access
- Route Tables (public and private)

Optional (Cost Saving)

- VPC Endpoints (S3, ECR, etc.) to reduce NAT costs for AWS service access

2. Create the VPC and Subnets (AWS CLI Example)

```
# Create VPC
aws ec2 create-vpc --cidr-block 10.0.0.0/16 \
 --tag-specifications 'ResourceType=vpc,Tags=[{Key=Name,Value=taskflow-vpc}]'

# Create public subnet in us-east-1a
aws ec2 create-subnet --vpc-id vpc-xxxxxexx \
 --cidr-block 10.0.1.0/24 --availability-zone us-east-1a \
 --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=public-1a}]'
# Create private app subnet in us-east-1a
aws ec2 create-subnet --vpc-id vpc-xxxxxexx \
 --cidr-block 10.0.11.0/24 --availability-zone us-east-1a \
 --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=private-app-1a}]'
```

3. Route Table Configuration

Public Route Table (public subnets)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	Internet Gateway (IGW)

Private Route Table (private subnets)

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	NAT Gateway (in same AZ)

4. Verify Routing

- Launch a test EC2 instance in a private subnet.
- Attach an IAM role with SSM (no SSH needed).
- Use SSM Session Manager to connect.
- Test internet connectivity: `curl https://www.google.com`
- Test AWS service access (e.g. S3): `aws s3 ls`

5. Expected Results

- Public subnets can access the internet.
- Private subnets can access the internet via NAT gateway.
- Private subnets can access AWS services (or via VPC endpoints).
- Routes are correctly associated with each subnet.
- No direct internet access to private subnets.

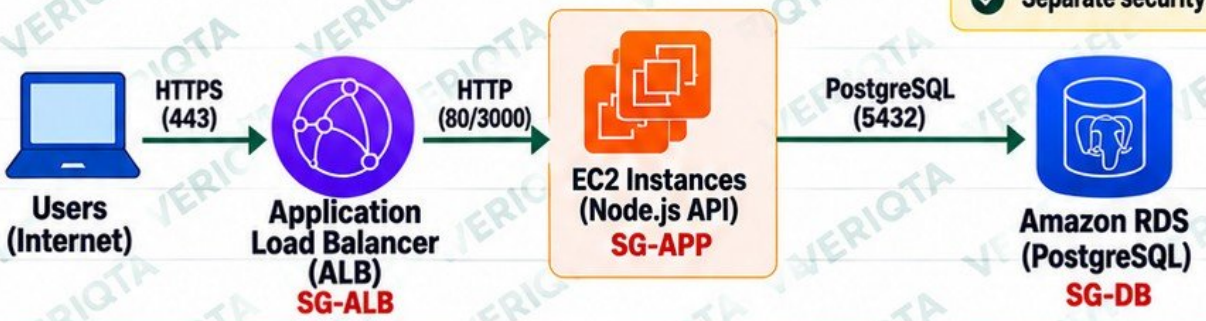
Security Groups and IAM

We will create least-privilege **security group** rules between the ALB, application servers, and database. We will also use **IAM roles** and **Systems Manager** to access and manage our EC2 instances instead of exposing SSH or embedding AWS keys.

Key Principles

- Allow only required traffic
- Use least privilege
- No public SSH access
- Use IAM roles (not AWS keys)
- Use Systems Manager for access
- Separate security groups per tier

1. Architecture and Traffic Flow



2. Security Group Rules (Least Privilege)

ALB Security Group (SG-ALB)

Type	Port	Source	Purpose
Inbound	443	0.0.0.0/0	HTTPS from internet
Inbound	80	0.0.0.0/0	(optional) Redirect to 443
Outbound	80/3000	SG-APP	Send traffic to application tier only

Application Security Group (SG-APP)

Type	Port	Source	Purpose
Inbound	80/3000	SG-ALB	Traffic from ALB only
Outbound	5432	SG-DB	Connect to database only
Outbound	443	0.0.0.0/0	OS updates (via NAT/VPCE)

Database Security Group (SG-DB)

Type	Port	Source	Purpose
Inbound	5432	SG-APP	PostgreSQL from application tier only
Outbound	All	0.0.0.0/0	(optional) For updates or backups

3. IAM Roles and Systems Manager

EC2 IAM Role (Least Privilege)

- Attach a role to EC2 instances (instead of using AWS keys).
- Allow only required permissions (e.g. CloudWatch, S3, ECR).
- Use IAM policies with the principle of least privilege.
- Rotate permissions by updating the role (no key rotation needed).

Systems Manager for Access

- Use AWS Systems Manager Session Manager to access EC2.
- No SSH keys, no open ports.
- Works over AWS API (private).
- Audit logs and access controls.
- More secure and easier to manage than SSH.

Avoid These (Common Mistakes)

- Opening SSH (port 22) to the internet.
- Using 0.0.0.0/0 for database access.
- Embedding AWS keys in code.
- Using overly permissive IAM policies (e.g. AdministratorAccess).
- Allowing all traffic between tiers.

4. Example IAM Policy for EC2 (Minimal)

```
{
   "Version": "2012-10-17",
   "Statement": [
      {
         "Effect": "Allow",
         "Action": [
            "ssm:UpdateInstanceInformation",
            "ssm:StartSession",
            "ssm:SendCommand",
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
            "logs:PutLogEvents"
         ],
         "Resource": "*"
      }
}
```

5. Verify Security and Access

- Confirm ALB is publicly accessible on port 443.
- Confirm EC2 instances are NOT publicly accessible (no SSH).
- Use Session Manager to connect to EC2.
- Confirm RDS is in a private subnet and not publicly accessible.
- Test application → database connectivity.
- Review security group flow and restrict any unnecessary rules.

6. Key Takeaways

- Use security groups to control network access between tiers.
- Follow least-privilege principles.
- Use IAM roles instead of AWS keys.
- Use Systems Manager for secure access.
- Keep the database private.
- Allow only the ports and sources required.
- Verify the setup before deploying the application.

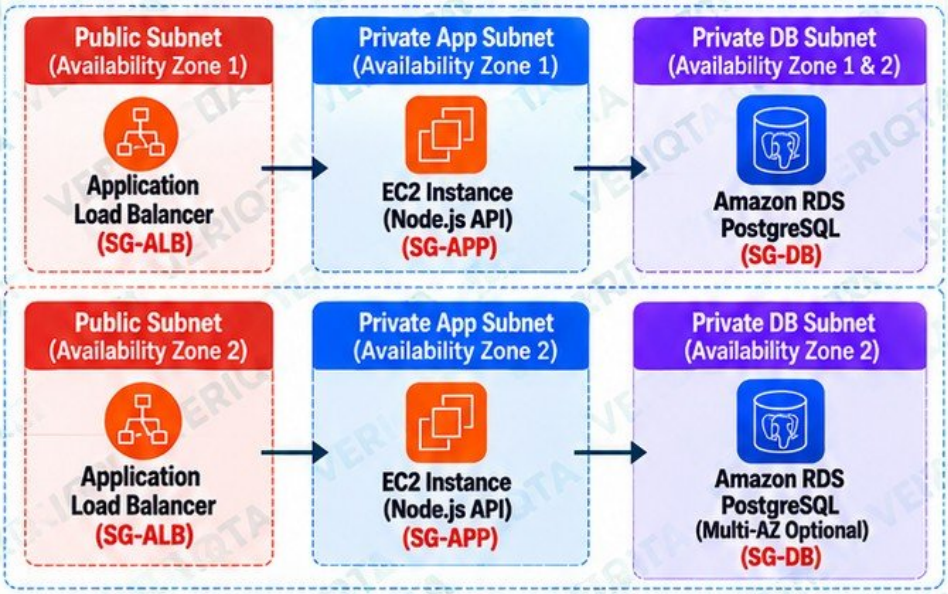
Create the PostgreSQL Database

In this step, we provision a private **Amazon RDS PostgreSQL** database, configure a DB subnet group, create the tasks schema, and set up backups, encryption, credentials, and connection testing.

Key Design Points

- RDS in private subnets (no public access)
- Accessible only from application tier
- Use a DB subnet group across 2 AZs
- Encrypt data at rest and in transit
- Enable automated backups
- Use strong credentials (or Secrets Manager)
- Choose Multi-AZ for high availability
- Test connectivity from application servers

1. RDS Architecture in Our VPC



RDS Configuration

Engine: PostgreSQL
Version: 16.x (or latest)
Deployment: Single-AZ (lab) / Multi-AZ (prod)
Instance class: db.t3.micro (lab)
Storage: 20 GB (GP3)
Subnet group: Private subnets (2 AZs)
Public access: No
Encryption: Enabled (KMS)
Backup retention: 7–30 days
Monitoring: Enhanced monitoring (optional)

2. Create DB Subnet Group

Create a DB Subnet Group with the two private database subnets from different Availability Zones.

```
# Create DB subnet group
aws rds create-db-subnet-group \
 --db-subnet-group-name taskflow-db-subnet \
 --db-subnet-group-description "TaskFlow DB
 DB subnets" \
  --subnet-ids subnet-0abc123 subnet-0def456
```

3. Create RDS PostgreSQL

Provision the RDS instance using your DB subnet group and secure settings.

```
aws rds create-db-instance \
  --db-instance-identifier taskflow-db \
  --engine postgres \
  --engine-version 16.3 \
  --db-instance-class db.t3.micro '
  --allocated-storage 20
  --db-subnet-group-name taskflow-db-subnet \
  --vpc-security-group-ids sg-xxxxxxxxx \
  --no-publicly-accessible ' \
  --storage-encrypted\
  --backup-retention-period 7
  --master-username postgres
  --master-user-password 'YourSecurePassword123!"
```

4. Create the Tasks Schema

Connect to the database and create the tasks table.

```
CREATE DATABASE taskflow;
\c taskflow;

CREATE TABLE tasks (
   id SERIAL PRIMARY KEY,
   title VARCHAR(255) NOT NULL,
   description TEXT,
   status VARCHAR(50) DEFAULT 'todo',
   created_at TIMESTAMP DEFAULT NOW(),
   updated_at TIMESTAMP DEFAULT NOW()
);

INSERT INTO tasks (title, description, status)
VALUES ('Learn AWS', 'Build a 3-tier app', 'todo');
```

5. Security Group for RDS (SG-DB)

- Type: PostgreSQL
- Port: 5432
- Source: SG-APP (application tier only)
- No inbound access from the internet
- Use separate security groups for each tier

6. Connection Testing

- Connect from an EC2 instance in the app subnet.
- Use the RDS endpoint and port 5432.
- Verify you can run queries on the tasks table.

```
psql -h taskflow-db.xxxxx.rds.amazonaws.com \
   -U postgres -d taskflow\
   -c "SELECT * FROM tasks;"
```

7. Backups and Encryption

- Enable automated backups.
- Set backup retention (e.g. 7–30 days).
- Enable encryption at rest (KMS).
- Consider Multi-AZ for production.

8. Managing Credentials

- Use AWS Secrets Manager (recommended).
- Do not hardcode credentials in your code.
- Grant IAM role access to read the secrett.
- Use environment variables in your application.

Example Secret (JSON)

```
{
   "username": "postgres",
   "password": "YourSecurePassword123!",
   "engine": "postgres",
   "host": "taskflow-db.xxxxx.rds.amazonaws.com",
   "port": 5432,
   "dbname": "taskflow"
}
```

9. Verify and Check

- RDS is in private subnets.
- No public access.
- Security group allows only SG-APP.
- Backups are enabled.
- Encryption is enabled.
- You can connect and run queries.

Build the Node.js API

In this step, we create an **Express** application with **GET, POST, PATCH, and DELETE** endpoints. We **validate inputs**, use **parameterized SQL queries**, and return meaningful **HTTP status codes**.

Key Objectives

- Build a REST API with Express
- Implement CRUD endpoints
- Validate input data
- Use parameterized SQL queries
- Return correct HTTP status codes
- Handle errors properly

1. Project Structure

```
taskflow-api/
├── src/
│   ├── routes/
│   │   └── tasks.js
│   ├── db.js
│   └── app.js
├── .env
├── package.json
└── package-lock.json
```

- routes/tasks.js contains all task endpoints
- db.js handles the PostgreSQL connection
- app.js sets up Express and middleware

2. Install Dependencies

```
mkdir taskflow-api && cd taskflow-api
npm init -y
npm install express pg dotenv cors express-validator
npm install --save-dev nodemon
```

3. Database Connection (db.js)

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  ssl: { rejectUnauthorized: false }
});

module.exports = pool;
```

- Uses environment variables
- Connection pool for better performance
- SSL enabled for RDS
- Credentials stored in .env (not in code)

4. Set Up Express App (app.js)

```
const express = require('express');
const cors = require('cors');
const { json } = require('body-parser');
const tasksRoutes = require('./routes/tasks');

const app = express();
app.use(cors());
app.use(json());

app.get('/health', (req, res) => {
  res.status(200).json({ message: 'API is running'});
});

app.use('/api/tasks', tasksRoutes);

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log('Server running
on port ${PORT}'));
```

- Enable CORS for frontend access
- Use JSON middleware
- Health check endpoint
- Register task routes

5. Task Routes (routes/tasks.js)

```
const express = require('express');
const { body, param, validationResult } =
  require('express-validator');
const pool = require('../db');
const router = express.Router();
```

- Input validation using express-validator
- Parameterized queries (protects against SQL injection)

6. GET /api/tasks

Get all tasks

```
router.get('/', async (req, res) => {
  try {
    const result = await pool.query(
      'SELECT * FROM tasks
      ORDER BY created_at DESC'
    );
    res.status(200).json(result.rows);
  } catch (err) {
    console.error(err);
    res.status(500).json({
      error: 'Failed to fetch tasks'
    });
  }
});
```

7. POST /api/tasks

Create a new task

```
router.post('/',
  body('title').notEmpty(),
  body('description').optional(),
  async (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({
        errors: errors.array()
      });
    }
    const { title,description } = req.body;
    const result = await pool.query(
      'INSERT INTO tasks (title, description,
      status) VALUES ($1, $2, $3)
      RETURNING *',
      [title, description, 'todo']
    );
    res.status(201).json(result.rows[0]);
});
```

8. PATCH /api/tasks/:id

Update a task

```
router.patch('/:id',
  param('id').isInt(),
  body('status',optional().isIn(
    ['todo','in_progress','done']),
  async (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({
        errors: errors.array()
      });
    }
    const { id } = req.params;
    const { status } = req.body;
    const result = await pool.query(
      'UPDATE tasks SET status = $1,
      updated_at = NOW() WHERE id = $2
      RETURNING *',
      [status, id]
    );
),res.status(200).json(result.rows[0]);
```

9. DELETE /api/tasks/:id

Delete a task

```
router.delete('/:id',
  param('id').isInt(),
  async (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({
        errors: errors.array()
      });
    }
    const { id } = req.params;
    const result = await pool.query(
      'DELETE FROM tasks WHERE id = $1
      RETURNING *',
      [id]
    );
    if (result.rows.length === 0) {
      return res.status(404).json({
        error: 'Task not found'
      });
),res.status(200).json({message: 'Task deleted'});
```

10. HTTP Status Codes Used

200	Success (GET, PATCH, DELETE)
201	Resource created (POST)
400	Bad request (validation error)
404	Resource not found
500	Internal server error

11. Input Validation Example

```
  const { body, validationResult } =
    require('express-validator');
}
  body('title'),notEmpty().withMessage('Title
  is required'),
  body('status',isIn(['todo','in_progress','done'])
    .withMessage('Invalid status')
```

12. Test with Postman or curl

```
# Get all tasks
curl -X GET http://localhost:3000/api/tasks

# Create a task
curl -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d "{"title":"Learn AWS","description":"Build 3-tier app"}"
```

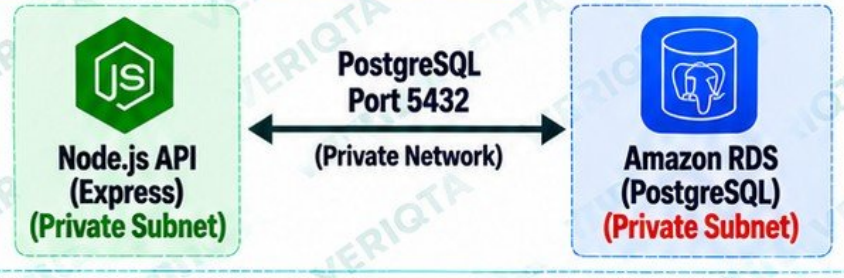
Connect the API to RDS

In this step, we store **credentials securely**, connect to **Amazon RDS PostgreSQL** using a **connection pool**, test database persistence, and investigate **authentication** and **connectivity failures**.

Key Objectives

- Store database credentials securely
- Use a PostgreSQL connection pool
- Test database persistence (CRUD)
- Handle connection and auth failures
- Use environment variables / Secrets Manager
- Follow security best practices

1. Architecture: API to RDS Connection



Important

- RDS is in a private subnet
- Only allow traffic from the API security group
- Use IAM/Secrets Manager or environment variables for credentials
- Do NOT hardcode credentials in code
- Use SSL for production

2. Store Credentials Securely

Option 1: AWS Secrets Manager (Recommended)

- Store DB username and password
- Use IAM role for EC2 to access the secret
- Rotate secrets if needed

Example secret (JSON):

```
{
  "username": "postgres",
  "password": "YourSecurePassword123!",
  "host": "taskflow-db.xxxxx.rds.amazonaws.com",
  "port": 5432,
  "database": "taskflow"
}
```

Option 2: Environment Variables (for labs)

- Store in .env file (not in code)
- Use different values for dev, staging, prod
- Keep .env out of GitHub (use .gitignore)

.env example:

```
DB_HOST=taskflow-db.xxxxx.rds.amazonaws.com
DB_PORT=5432
DB_NAME=taskflow
DB_USER=postgres
DB_PASSWORD=YourSecurePassword123!
```

3. PostgreSQL Connection Pool (db.js)

db.js (using pg)

```
const { Pool } = require('pg');

const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  database: process.env.DB_NAME,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  max: 10,              // maximum connections
  idleTimeoutMillis: 30000,      // 30 seconds
  connectionTimeoutMillis: 10000, // 10 seconds
  ssl: { rejectUnauthorized: false } // use true in prod
});
module.exports = pool;
```

4. Use the Connection in Routes (Example)

routes/tasks.js

```
const express = require('express');
const router = express.Router();
const pool = require('../db');

router.get('/', async (req, res) => {
  try {
    const result = await pool.query(
      'SELECT id, title, description, status, created_at
       FROM tasks ORDER BY created_at DESC'
    res.json(result.rows);
  } catch (err) {
    console.error('Database query error:', err);
    res.status(500).json({ error: 'Failed to fetch tasks' });
}});
module.exports = router;
```

5. Test Database Persistence

- Start the API
- Create a task (POST)
- Read tasks (GET)
- Update a task (PATCH)
- Delete a task (DELETE)
- Verify data in RDS

Example: Create a Task (POST)

```
curl -X POST http://localhost:3000/api/tasks \
  -H "Content-Type: application/json" \
  -d '{
    "title": "Learn AWS",
    "description": "Build a 3-tier app",
    "status": "todo"
```

6. Verify in RDS

Connect to RDS (from a bastion or EC2 in the same VPC)

```
psql -h taskflow-db.xxxxx.rds.amazonaws.com \
  -U postgres -d taskflow

-- Check tasks table
SELECT * FROM tasks;

 id | title       | status |
----+-------------+--------+-
  1 | Learn AWS   | todo   |
(1 row)
```

7. Common Errors and Troubleshooting

Error	Cause	Fix
ECONNREFUSED	Wrong endpoint or security group	Check RDS endpoint and SG rules
Authentication failed	Wrong username/password	Verify credentials in Secrets Manager or .env
timeout expired	Network or firewall issue	Ensure API and RDS are in same VPC and SG allows port 5432
no pg_hba.conf entry	RDS not allowing the connection	Check security group and parameter group
SSL error	SSL configuration issue	Use ssl: { rejectUnauthorized: false } (in production use proper SSL)

8. Best Practices

- Store credentials in AWS Secrets Manager or environment variables
- Use IAM roles (if using Secrets Manager)
- Use a connection pool (avoid creating a new connection per request)
- Enable SSL for production
- Restrict RDS access to the application security group only
- Use least privilege database user
- Monitor connection count and performance
- Handle and log errors properly
- Do not expose RDS to the internet

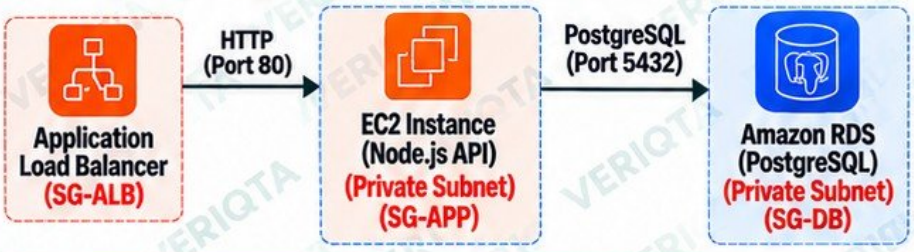
Deploy the Backend on EC2

In this step, we package the **Node.js API**, launch a **private EC2 instance** with an **IAM role**, run the application using **systemd**, and verify service health and startup logs.

Key Objectives

- Package the application for deployment
- Launch a private EC2 instance
- Attach an IAM role (use SSM, no SSH)
- Install dependencies and deploy the API
- Run the API using systemd
- Verify service health and startup logs
- Troubleshoot common issues

1. Backend Deployment Architecture



2. Launch a Private EC2 Instance

- Use Amazon Linux 2023 (or Ubuntu).
- Place in a private subnet (no public IP).
- Attach an IAM role for SSM and required AWS permissions.
- Use a security group that allows inbound traffic only from the ALB (port 3000).
- Use Session Manager (SSM) to connect (no SSH).

3. Package the Application

Create a deployment package and upload to the EC2 instance using SSM or from a private S3 bucket.

```
# On your local machine
cd taskflow-api
npm install --production
zip -r taskflow-api.zip .

# Upload using AWS CLI (SSM - no SSH)
aws ssm send-command \
  --instance-ids i-0xxxxxxxxxxxxxxxx \
  --document-name "AWS-RunShellScript"
  --parameters commands="mkdir -p /home/ec2-user/app && \
  curl -o /home/ec2-user/app/taskflow-api.zip https://s3.amazonaws.com/\
  your-bucket/taskflow-api.zip && unzip -o /home/ec2-user/app/taskflow-api.zip \
  -d /home/ec2-user/app"
```

4. Install Dependencies on EC2

Connect via Session Manager and install Node.js and required packages.

```
# Update and install Node.js (Amazon Linux 2023)
sudo dnf update -y
sudo dnf install -y nodejs npm

# Go to application directory
cd /home/ec2-user/app
npm install --production

# Verify installation
node -v
npm -v
```

5. Create a systemd Service

Run the API using systemd so it starts automatically and restarts on failure.

```
sudo tee /etc/systemd/system/taskflow-api.service << 'EOF'
[Unit]
Description=TaskFlow Node.js API
After=network.target
[Service]
Type=simple
User=ec2-user
WorkingDirectory=/home/ec2-user/app
ExecStart=/usr/bin/node app.js
Restart=always
Environment=NODE_ENV=production
[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable taskflow-api
sudo systemctl start taskflow-api
```

6. Verify Service Health

Check the service status, logs, and confirm the API is running on port 3000.

```
# Check service status
sudo systemctl status taskflow-api

# View recent logs
sudo journalctl -u taskflow-api -n 50 --no-pager

# Test locally on the instance
curl -I http://localhost:3000/api/health

# Expected response
HTTP/1.1 200 OK
Content-Type: application/json
{"status":"ok","message":"TaskFlow API is running"}
```

7. Security Best Practices

- Keep the EC2 instance in a private subnet.
- Do not assign a public IP.
- Use an IAM role instead of AWS keys.
- Use SSM for all access (no SSH).
- Restrict inbound traffic to only the ALB security group.
- Keep the OS and Node.js packages updated.
- Store environment variables in AWS Systems Manager or Secrets Manager.

8. Common Issues and Troubleshooting

Issue	Cause	Fix
Service not starting	Missing dependencies or wrong path	Check logs with journalctl -u taskflow-api
Cannot connect to DB	Security group / network configuration	Verify SG rules and VPC routing
Permission denied	Wrong file permissions or user	Use correct user and chmod
Port not accessible	App not running or wrong port	Check systemd status and application logs
App crashes on restart	Environment variable missing	Set NODE_ENV and DB configs

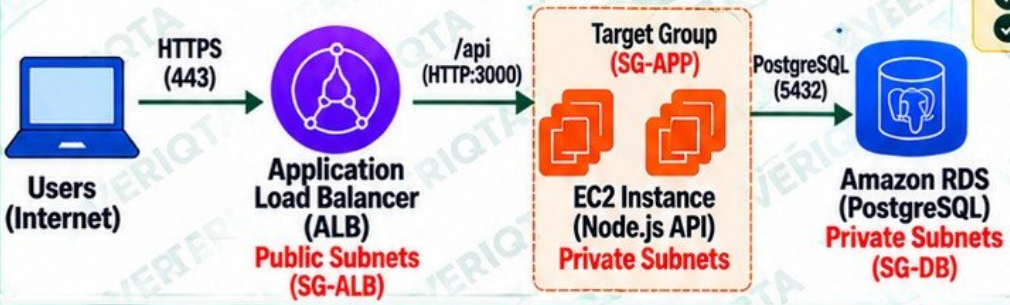
Add the Application Load Balancer

In this step, we create a **public HTTPS-enabled Application Load Balancer (ALB)**, configure a target group with **health checks**, route **/api** requests to the **backend EC2 instance**, and diagnose **unhealthy targets.**

Key Objectives

- Create a public ALB with HTTPS (SSL)
- Create a target group for the backend
- Configure health checks
- Route /api requests to the backend
- Use a valid SSL certificate (ACM)
- Diagnose and fix unhealthy targets

1. Architecture with ALB



2. Create an Application Load Balancer

- Type: Application Load Balancer (ALB)
- Scheme: Internet-facing
- Listeners: HTTPS (443)
- Subnets: Public subnets (2 AZs)
- Security group: Allow 443 from anywhere (0.0.0.0/0)
- Use ACM certificate for your domain (or test with self-signed)

```
# Create ALB using AWS CLI
aws elbv2 create-load-balancer \
  --name taskflow-alb
  --subnets subnet-12345678 subnet-87654321
  --security-groups sg-alb123
  --type application
  --scheme internet-facing
  --region us-east-1
```

3. Create a Target Group

- Target type: Instances (EC2)
- Protocol: HTTP
- Port: 3000
- Health check path: /api/health
- Healthy threshold: 2
- Unhealthy threshold: 2
- Timeout: 5 seconds
- Interval: 30 seconds

```
# Create target group
aws elbv2 create-target-group \
  --name taskflow-tg
  --protocol HTTP
  --port 3000
  --vpc-id vpc-0abc123def456
  --target-type instance
  --health-check-path /api/health
  --health-check-interval-seconds 30
  --health-check-timeout-seconds 5
  --healthy-threshold-count 2
  --unhealthy-threshold-count 2
```

4. Configure ALB Listener and Rules

- Listener: HTTPS (443) with ACM certificate
- Default action: Fixed response or redirect
- Add rule for /api path to forward to target group
- Optional: Redirect HTTP (80) to HTTPS (443)

```
# Create HTTPS listener
aws elbv2 create-listener \
  --load-balancer-arn arn:aws:elasticloadbalanc
ing:....:loadbalancer/app/taskflow-alb/123abc \
  --protocol HTTPS
  --port 443
  --certificates CertificateArn=arn:aws:acm:...:
```

```
# Add rule for /api
aws elbv2 create-rule \
  --listener-arn arn:aws:elasticloadbalancing:..:
  --priority 10
  --conditions Field=path-pattern,Values=/api/*
  --actions Type=forward,TargetGroupArn=arn:aws:
elasticloadbalancing:...:targetgroup/taskflow-tg
```

5. Register Targets (EC2 Instances)

- Register your EC2 instance(s) with the target group
- Ensure the instance is running the Node.js API on port 3000
- Instances must be in private subnets
- Security group must allow traffic from ALB (port 3000)

```
# Register an EC2 instance
aws elbv2 register-targets \
  --target-group-arn arn:aws:elasticloadbalancing:...:
targetgroup/taskflow-tg/123abc
  --targets Id=i-0abc123def456
```

6. Health Check Endpoint

- Create a simple /api/health endpoint in your Node.js app
- Return HTTP 200 when the service is healthy
- ALB uses this endpoint to check instance health

```
// routes/health.js
app.get('/api/health', (req, res) => {
  res.status(200).json({
    status: 'ok',
    message: 'TaskFlow API is healthy',
    timestamp: new Date().toISOSTring()
  });
})
```

7. Test the Load Balancer

- Access the ALB DNS name or your domain
- Test /api endpoints through the ALB
- Verify the target is healthy in the console

```
# Test via ALB DNS name
curl -I https://taskflow-alb-123456.us-east-
1.elb.amazonaws.com/api/health
```

```
# Expected response
HTTP/2 200
content-type: application/json

    "status": "ok",
    "message": "TaskFlow API is healthy"
```

8. Common Issues and Fixes

Issue	Cause	Fix
Target unhealthy	Health check failing	Check /api/health endpoint, app logs, SG rules
503 Service Unavailable	No healthy targets	Ensure instances are running and registered
Timeout	Security group blocking	Allow ALB SG to EC2 port 3000
SSL certificate error	Invalid or missing ACM cert	Use a valid ACM certificate (or self-signed for test)
Cannot access ALB	Wrong DNS or listener	Verify ALB DNS name and HTTPS listener
Request not routed	Incorrect listener rule	Check /api path rule and target group

9. Best Practices

- Use HTTPS with a valid ACM certificate
- Keep EC2 instances in private subnets
- Use health checks to automatically remove unhealthy instances
- Set appropriate health check intervals and thresholds
- Use multiple instances across 2 AZs for high availability
- Monitor ALB and target group metrics (CloudWatch)
- Enable access logs (optional)
- Use least-privilege security group rules

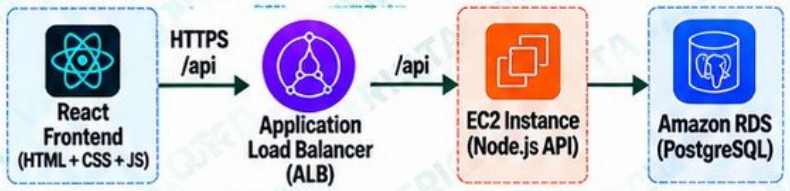
Build the React Frontend

In this step, we create a responsive TaskFlow interface using React with task forms, status updates, loading indicators, error handling, and integration with the backend API.

Key Objectives

- Create a React frontend for TaskFlow
- Build task forms (create, update, delete)
- Integrate with the backend API
- Show loading indicators and error messages
- Display and update task status
- Build a responsive and user-friendly UI

1. Frontend Architecture



2. Create a React Project

```
# Create a new React project
npm create vite@latest taskflow-frontend -- --template react
cd taskflow-frontend
npm install
npm install axios react-router-dom
```

3. Project Structure

```
taskflow-frontend/
├── src/
│   ├── components/
│   │   ├── TaskForm.jsx
│   │   ├── TaskList.jsx
│   │   ├── Loading.jsx
│   │   └── ErrorMessage.jsx
│   ├── pages/
│   │   └── Home.jsx
│   ├── services/
│   │   └── api.js
│   ├── App.jsx
│   └── main.jsx
├── package.json
└── vite.config.js
```

- Organize components for reusability
- Use a separate API service file
- Keep the UI responsive
- Handle loading and error states
- Use environment variables for the API URL

4. API Service (src/services/api.js)

```
import axios from 'axios';

const API_BASE_URL = import.meta.env.VITE_API_URL ||
  'https://your-domain.com/api';

const api = axios.create({
  baseURL: API_BASE_URL,
  headers: { 'Content-Type': 'application/json' },
  timeout: 10000,
});

export const getTasks = () => api.get('/tasks');
export const createTask = (data) => api.post('/tasks', data);
export const updateTask = (id, data) => api.patch('/tasks/${id}', data);
export const deleteTask = (id) => api.delete('/tasks/${id}');

export default api;
```

5. Task Form Component (TaskForm.jsx)

```
import { useState } from 'react';
import { createTask } from '../services/api';

function TaskForm({ onTaskCreated }) {
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      setLoading(true);
      await createTask({ title, description, status: 'todo' });
      setTitle('');
      setDescription('');
      onTaskCreated();
    } catch (err) {
      setError(err.response?.data?.message || 'Failed to create task');
    } finally {
      setLoading(false);
    }
  };

  return (
    <form onSubmit={handleSubmit} className="task-form">
      <input value={title} onChange={(e) => setTitle(e.target.value)}
        placeholder="Task title" required />
      <textarea value={description} onChange={(e) => setDescription(e.
        target.value)}/>
      <button type="submit" disabled={loading}>{loading ?
                                  "Creating..." : "Add Task"}</button>
      {error && <p className="error">{error}</p>}
    </form>
  );
}

export default TaskForm;
```

6. Task List with Status Update (TaskList.jsx)

```
import { useEffect, useState } from 'react';
import { getTasks, updateTask, deleteTask } from '../services/api';

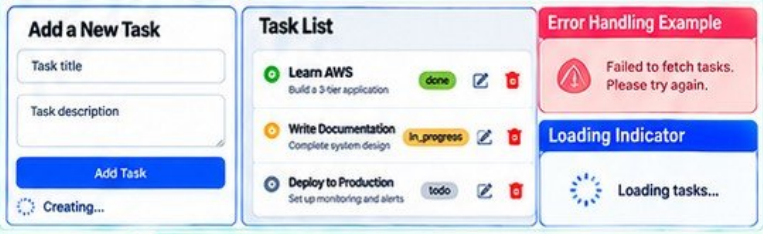
function TaskList() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');

  useEffect(() => {
    const fetchTasks = async () => {
      try {
        const res = await getTasks();
        setTasks(res.data);
      } catch (err) {
        setError('Failed to fetch tasks');
      } finally {
        setLoading(false);
      }
    };
    fetchTasks();
  }, []);
  // Update status and delete functions (omitted for brevity)
  // Render tasks with loading and error handling
export default TaskList;
```

Features Implemented

- Display list of tasks from API
- Update task status (todo, in_progress, done)
- Delete tasks
- Show loading indicator
- Handle and display errors
- Responsive and clean UI

7. Sample React Frontend (TaskFlow UI)



8. Environment Variables

```
# .env
VITE_API_URL=https://your-domain.com/api
```

- Store API URL in .env file
- Use different values for development and production

9. Best Practices

- Use environment variables for API endpoints
- Handle loading and error states
- Validate user input on the frontend
- Keep the UI responsive (mobile and desktop)
- Use meaningful HTTP status messages from the API
- Follow a clean and modular component structure

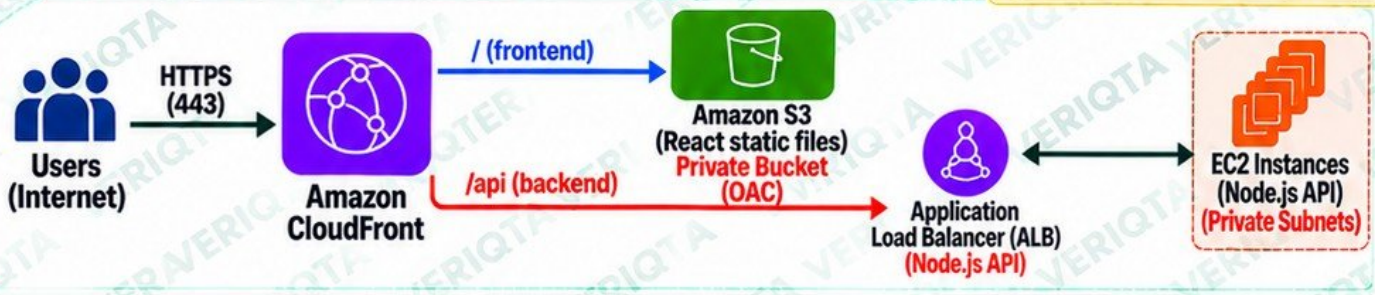
Publish with S3 and CloudFront

In this step, we host the compiled React assets in a private S3 bucket, serve them through CloudFront using Origin Access Control (OAC), and configure HTTPS and API path routing.

Key Objectives

- Host React assets in private S3
- Use CloudFront with Origin Access Control
- Configure HTTPS with a valid certificate
- Route frontend (/) to S3 via CloudFront
- Route /api to the backend ALB
- Test the application and verify routing

1. Architecture Overview



2. Build the React Application

- Install dependencies
- Build the production assets
- The output will be in the dist/ or build/ folder

```
# Install dependencies
npm install

# Build for production
npm run build

# Output folder (example)
ls -l dist/
```

3. Create a Private S3 Bucket

- Create an S3 bucket (private)
- Block all public access
- Enable versioning (optional)
- Allow CloudFront access using Origin Access Control (OAC)

```
# Create S3 bucket
aws s3 mb s3://taskflow-frontend

# Block public access
aws s3api put-public-access-block \
  --bucket taskflow-frontend \
  --public-access-block-configuration
BlockPublicAcls=true,IgnorePublicAcls=true, \
BlockPublicPolicy=true,RestrictPublicBuckets=true
```

4. Create CloudFront Distribution

- Set S3 as the origin with OAC
- Use a custom domain and HTTPS (ACM certificate)
- Add a second origin for the API (ALB)
- Create behaviors for / and /api

```
# Create Origin Access Control (OAC)
aws cloudfront create-origin-access-control \
  --origin-access-control-config \
  Name=taskflow-oac,
  OriginAccessControlOriginType=s3,
  SigningBehavior=always,
  SigningProtocol=sigv4

# Create CloudFront distribution
# (use AWS Console or Terraform for full setup)
```

5. CloudFront Behaviors (Path Routing)

- Behavior 1: / → S3 (React frontend)
- Behavior 2: /api/* → ALB (backend API)
- Use HTTPS only (redirect HTTP to HTTPS)
- Use a valid ACM certificate for your domain

Path Pattern	Origin	Purpose
/	S3 (OAC)	Serve React static files (index.html, assets)
/api/*	ALB (Node.js API)	Forward API requests to the backend
Default (*)	S3 (OAC)	Fallback to React app (for client-side routing)

6. S3 Bucket Policy (OAC)

- Allow only CloudFront distribution to access the bucket
- Keep the bucket private
- Do not allow public access

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::taskflow-frontend/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront
::123456789012:distribution/EDFDVBD6328HDSS"
        }
      }
    }
}
```

7. Configure HTTPS (ACM)

- Request a public certificate in AWS Certificate Manager
- Validate the domain (DNS or email)
- Attach the certificate to CloudFront
- Enforce HTTPS (redirect HTTP to HTTPS)

```
# Request certificate (use console or CLI)
aws acm request-certificate \
  --domain-name your-domain.com \
  --validation-method DNS \
  --region us-east-1  # CloudFront certs

# Attach the certificate to the
# CloudFront distribution (in console)
```

8. Test the Deployed Application

- Open your domain (https://your-domain.com)
- Verify the React frontend loads
- Test /api endpoints (e.g. https://your-domain.com/api/tasks)
- Check browser network tab for correct routing
- Verify HTTPS and valid certificate

9. Troubleshooting

Issue	Cause	Fix
React app not loading	S3 bucket not accessible via OAC	Check OAC and bucket policy
403 Access Denied	Incorrect bucket policy	Allow CloudFront distribution ARN
/api returns 404	Wrong path pattern	Ensure /api/* routes to ALB
Mixed content (HTTP)	Using HTTP resources	Use HTTPS for all resources
CORS error	ALB or API not allowing domain	Configure CORS on the API

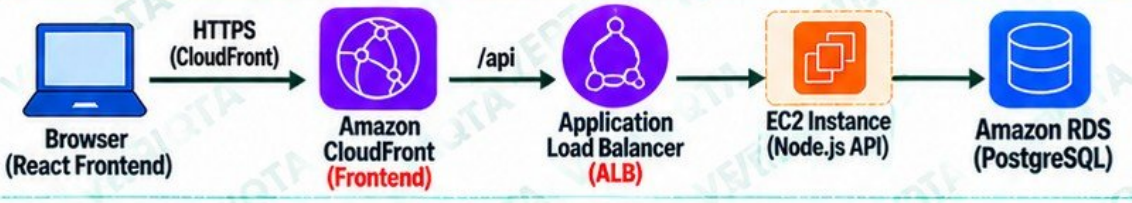
Test the Complete Application

In this step, we test the end-to-end TaskFlow application, from the browser to to the database. We verify CRUD persistence, handle invalid requests, check refresh behavior, test API failures, confirm CORS or same-origin configuration, and use browser developer tools for troubleshooting.

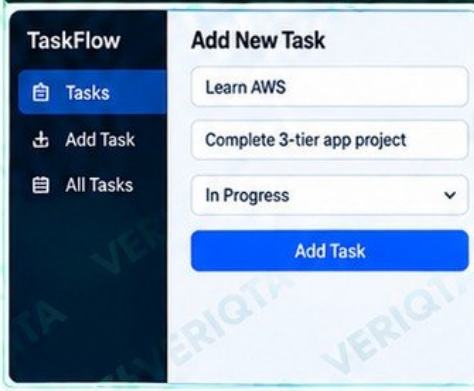
Key Objectives

- Test browser-to-database workflows
- Verify CRUD operations and persistence
- Test invalid requests and error handling
- Check refresh behavior (no data loss)
- Test API failures and recovery
- Validate CORS or same-origin configuration
- Use browser developer tools for debugging

1. End-to-End Architecture (What We Are Testing)



2. Test CRUD Operations (Frontend)



3. Verify Data Persistence

- Create a new task (POST)
- Verify it appears in the list (GET)
- Update the task (PATCH)
- Delete the task (DELETE)
- Confirm data is saved in RDS

```
-- Check data in PostgreSQL (RDS)
SELECT id, title, description,
   status, created_at
   FROM tasks
   ORDER BY created_at DESC;
 id |       title        |   status
----+--------------------+-------------
  1 | Learn AWS          | in_progress
  2 | Build 3-tier app   | todo
```

4. Test Invalid Requests

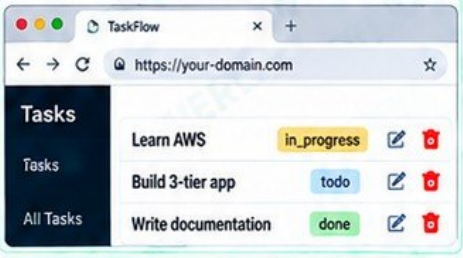
- Send missing or invalid data
- Use an invalid task ID
- Check API validation messages
- Verify correct HTTP status codes

```
# Example: Invalid request (missing title)
curl -X POST https://your-domain.com/api/tasks \
  -H "Content-Type: application/json" \
  -d "{"description":"test"}"
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
   "error": "Title is required"
}
```

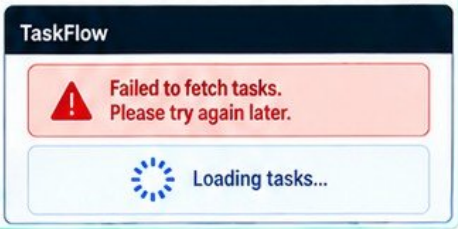
5. Test Refresh Behavior

- Create or update a task
- Refresh the browser (F5)
- Verify data is still there
- Ensure no data loss or duplicate entries



6. Test API Failures

- Stop the API service or block traffic
- Check frontend error messages
- Verify proper error handling and loading indicators
- Restore the API and test recovery



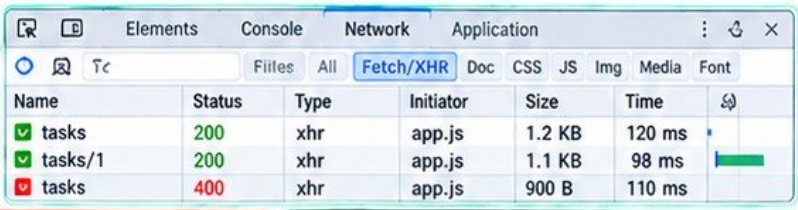
7. Validate CORS or Same-Origin

- Check for CORS errors in browser console
- Ensure the frontend domain is allowed in the API CORS configuration
- Use same-origin when possible (CloudFront → /api → ALB)



8. Use Browser Developer Tools

- Open DevTools (F12)
- Use the Network tab to inspect API requests
- Check request/response headers and payloads
- Use the Console tab to identify JavaScript errors
- Inspect application storage (Local Storage, Cookies)



9. Common Issues and Fixes

Issue	Cause	Fix
Data not saving	API or RDS issue	Check API logs and DB connection
CORS error	Domain not allowed	Add frontend domain to CORS config
404 on /api	Wrong path or routing	Verify ALB listener and route rules
Frontend not loading	S3/CloudFront issue	Check S3 files, CloudFront cache and OAC
500 Internal Server Error	Application error	Check API logs and fix code
Network error	Security group or NACL	Check SG rules and VPC routing

10. Success Checklist

- Frontend loads through CloudFront (HTTPS)
- All CRUD operations work correctly
- Data is persisted in RDS
- Invalid requests return proper error messages
- Refresh behavior works (no data loss)
- API failures show user-friendly errors
- No CORS errors (or using same-origin)
- All tests pass in browser developer tools

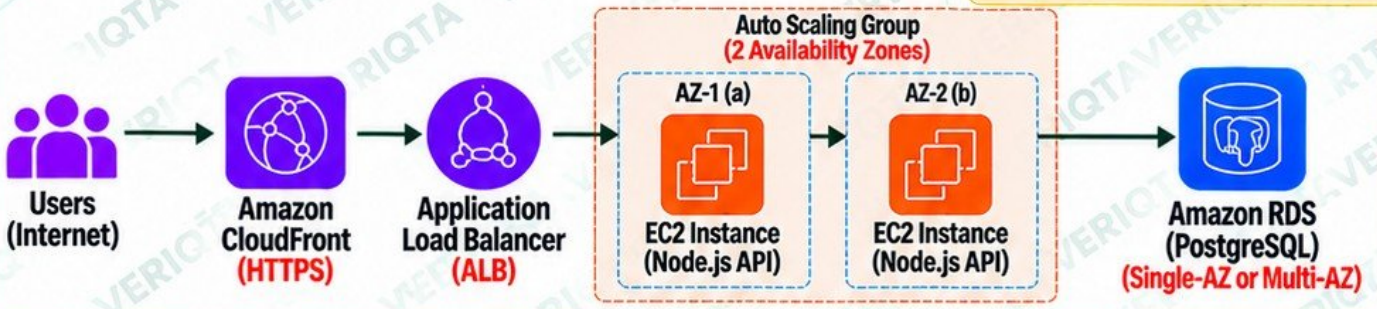
Make the Backend Highly Available

In this step, we create an **EC2 launch template** and **Auto Scaling group** across **two Availability Zones**, test replacement of an unhealthy instance, and explain the limits of a **single-AZ database**.

Key Objectives

- Create an EC2 launch template
- Create an Auto Scaling group (2 AZs)
- Use the ALB for health checks
- Test replacement of an unhealthy instance
- Understand single-AZ vs Multi-AZ database
- Know the limitations of a single-AZ database

1. High Availability Architecture



2. Create an EC2 Launch Template

- Use Amazon Linux 2023 (or Ubuntu)
- Install Node.js and dependencies
- Configure user data to start the API with systemd
- Attach an IAM role (no AWS keys)
- Open only required ports (e.g. 3000)
- Use the latest AMI and instance type (e.g. t3.micro)

```
# Create launch template
aws ec2 create-launch-template \
 --launch-template-name taskflow-api-lt \
 --version-description "v1" \
 --launch-template-data file://launch-template.json
```

3. User Data (Install and Run API)

- Update packages
- Install Node.js and app dependencies
- Copy application files (from S3 or bake in AMI)
- Enable and start the systemd service

```
# Example user data (bash)
#!/bin/bash
sudo dnf update -y
sudo dnf install -y nodejs npm
cd /home/ec2-user/app
npm install --production
sudo cp taskflow-api.service \
  /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable taskflow-api
sudo systemctl start taskflow-api
```

4. Create an Auto Scaling Group

- Use the launch template
- Deploy across 2 Availability Zones
- Attach to the ALB target group
- Set min, desired, and max capacity
- Use health checks (ALB)
- Enable instance replacement

```
# Create Auto Scaling Group
aws autoscaling create-auto-scaling-group \
 --auto-scaling-group-name taskflow-api-asg \
 --launch-template LaunchTemplateName=taskflow-api-lt,
  Version="$Latest" \
 --min-size 2 \
 --desired-capacity 2 \
 --max-size 4 \
 --vpc-zone-identifier subnet-az1,subnet-az2 \
 --target-group-arns arn:aws:elasticloadbalancing:
  region:123456789012:targetgroup/taskflow-tg/abc123 \
 --health-check-type ELB
```

5. Verify Instances in Two Availability Zones

- Check that instances are running in different AZs
- Verify they are registered with the ALB target group
- Confirm the API is healthy on both instances

EC2 Instances (Auto Scaling Group)

Instance ID	AZ	State	Status Check	Target Group
i-0abc123def	us-east-1a	Running	2/2 checks	Healthy
i-0def456ghi	us-east-1b	Running	2/2 checks	Healthy

6. Test Replacement of an Unhealthy Instance

- Stop or terminate one instance
- Verify Auto Scaling launches a new instance
- Confirm the new instance registers with the ALB
- Ensure the application remains available

```
# Simulate failure by stopping an instance
aws ec2 stop-instances --instance-ids i-0abc123def

# Check Auto Scaling activity
aws autoscaling describe-scaling-activities \
  --auto-scaling-group-name taskflow-api-asg \
  --max-items 5
```

Auto Scaling Activity

Time	Description	Status
12:10	Terminating unhealthy instance	Successful
12:11	Launching new instance	Successful
12:13	New instance registered	Successful

7. Single-AZ vs Multi-AZ Database

- Single-AZ: Data stored in one Availability Zone only
- More affordable but not highly available
- If the AZ fails, the database becomes unavailable
- Multi-AZ: Synchronous replica in a second AZ
- Automatic failover to standby
- Recommended for production workloads

8. Limitations of a Single-AZ Database

- No automatic failover
- Database becomes unavailable if the AZ fails
- Manual recovery or snapshot restore required
- Not suitable for high availability requirements
- Use Multi-AZ for production environments

9. Best Practices

- Use at least 2 AZs for the backend
- Use ALB health checks for instance health
- Keep the application stateless
- Store data in Multi-AZ RDS for production
- Set appropriate scaling policies
- Monitor with CloudWatch and alarms
- Test failure scenarios regularly

10. Common Issues and Fixes

Issue	Cause	Fix
New instance not launching	Incorrect ASG configuration	Check launch template, subnets, and capacity settings
Instance unhealthy	App not starting or wrong port	Check user data, systemd logs, and security group rules
ALB shows no healthy targets	Health check path misconfigured	Verify health check path (/api/health) and response code
Application downtime	Only one instance running	Set min capacity to 2 and use 2 AZs
Database unavailable	Single-AZ RDS and AZ failure	Use Multi-AZ RDS for high availability

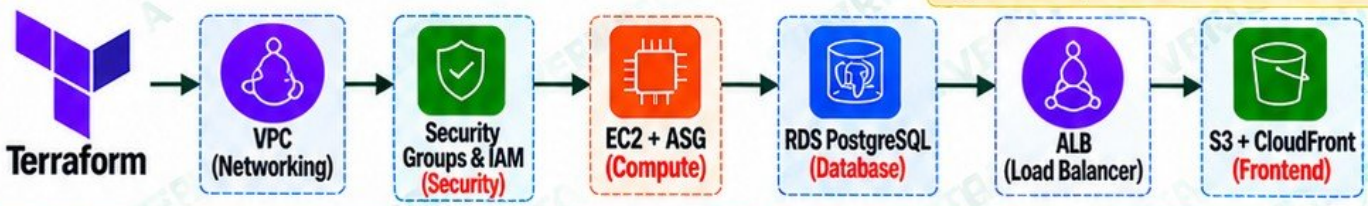
Infrastructure as Code with Terraform

In this step, we organize Terraform configuration for **networking**, **security**, **compute**, **database**, and **frontend delivery**. We use **variables**, **outputs**, **remote state protection**, and **plan reviews**.

Key Objectives

- Structure Terraform code for the entire application
- Create reusable modules (VPC, ALB, EC2, RDS, S3, CloudFront)
- Use variables and outputs
- Configure secure remote state in S3 with locking
- Run plan and review changes before apply
- Follow best practices for maintainable code

1. Architecture Overview (Terraform Configuration)



2. Project Structure

```
taskflow-terraform/
├── modules/
│   ├── vpc/
│   ├── security/
│   ├── compute/
│   ├── database/
│   ├── alb/
│   └── frontend/
├── envs/
│   ├── dev/
│   └── prod/
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars
├── versions.tf
└── backend.tf
```

- Use modules for reusability
- Separate environments (dev/prod)
- Keep state files in remote backend
- Use meaningful naming and tagging
- Follow consistent folder structure

3. Example: VPC Module (modules/vpc/main.tf)

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags = {
    Name = "${var.project}-vpc"
  }
}
resource "aws_subnet" "public" {
  count             = length(var.public_subnets)
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnets[count.index]
  availability_zone = var.azs[count.index]
  map_public_ip_on_launch = true
  tags = {
    Name = "${var.project}-public-${count.index + 1}"
  }
}
```

4. Using Variables (variables.tf)

```
variable "project" {
  description = "Project name"
  type        = string
}

variable "vpc_cidr" {
  description = "VPC CIDR block"
  type        = string
}

variable "azs" {
  description = "Availability zones"
  type        = list(string)
  default     = ["us-east-1a",
                 "us-east-1b"]
}
```

5. Outputs (outputs.tf)

```
output "vpc_id" {
  description = "VPC ID"
  value       = aws_vpc.main.id
}
output "alb_dns_name" {
  description = "ALB DNS name"
  value       = aws_lb.taskflow_alb.dns_name
}

output "rds_endpoint" {
  description = "RDS endpoint"
  value       = aws_db_instance.taskflow.endpoint
}
```

6. Remote State in S3 (backend.tf)

```
terraform {
  backend "s3" {
    bucket         = "taskflow-terraform-state"
    key            = "prod/terraform.tfstate"
    region         = "us-east-1"
    encrypt        = true
    dynamodb_table = "taskflow-terraform-locks"
  }}
```

- Store state in S3 (encrypted)
- Use DynamoDB for state locking
- Never commit state files to Git
- Use separate state for each environment

7. Plan and Apply (CLI)

```
# Initialize Terraform
terraform init

# Review the execution plan
terraform plan -var-file="prod/terraform.tfvars"

# Apply after review
terraform apply -var-file="prod/terraform.tfvars"

# Destroy (only when needed)
terraform destroy -var-file="prod/terraform.tfvars"
```

8. Modules for Each Component

VPC Module	Security Module	Compute Module	Database Module	ALB Module	Frontend Module
Networking (VPC, Subnets, Route Tables)	Security Groups, IAM Roles, NACLs	Launch Template, Auto Scaling Group, EC2 (Node.js API)	RDS PostgreSQL, Subnet Group, Parameter Group	Application Load Balancer, Target Group	S3 (private), CloudFront, ACM, Route 53

9. Plan Review Example

```
Terraform will perform the following actions:
  # aws_instance.app[0] will be created
  + resource "aws_instance" "app" {
      + id            = (known after apply)
      + instance_type = "t3.mirco"
      + tags          = {
          + "Name" = "taskflow-api"
```

10. Common Issues and Fixes

Issue	Cause	Fix
State file not found	Backend not configured	Configure S3 backend and run terraform init
Resource already exists	Duplicate configuration	Import existing resource or use unique names
Permission denied	Missing IAM permissions	Attach correct IAM policy to your user/role
Plan shows unexpected changes	Inconsistent state or manual changes	Review state, use terraform refresh
State lock failure	DynamoDB table not set	Configure DynamoDB for state locking

11. Best Practices

- Use modules and reusable code
- Keep environments separate (dev/prod)
- Use remote state with encryption and locking
- Use meaningful tags and naming conventions
- Review the plan before applying
- Use version control (Git)
- Limit Terraform permissions (least privilege)
- Document your infrastructure

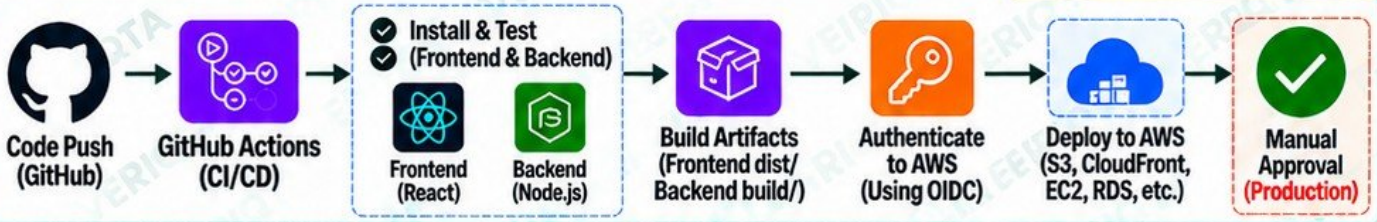
Build a GitHub Actions Pipeline

In this step, we create a GitHub Actions pipeline that tests the frontend and backend, checks dependencies, builds artifacts, authenticates to AWS with OIDC, and deploys after approval.

Key Objectives

- Set up a complete CI/CD pipeline
- Test frontend and backend
- Check dependencies and security
- Build artifacts
- Authenticate to AWS using OIDC
- Deploy after manual approval
- Follow best practices for reliable pipelines

1. Pipeline Architecture



2. Workflow File Structure

```
.github/
  workflows/
    deploy.yml
frontend/
  package.json
  tests/
backend/
  package.json
  tests/
infra/
  terraform/
```

- Keep workflow files in .github/workflows
- Use separate jobs for clarity
- Test both frontend and backend
- Build artifacts and upload
- Use environments for approval
- Deploy only after manual approval

3. GitHub Actions Workflow (deploy.yml)

```yaml
name: Deploy TaskFlow
on:
  push:
    branches: [ main ]
  workflow_dispatch:
jobs:
  test-frontend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
      - name: Install dependencies
        run: npm ci
      - name: Run tests
        run: npm run test
      - name: Build frontend
        run: npm run build
      - name: Upload frontend artifact
        uses: actions/upload-artifact@v4
        with:
          name: frontend-dist
          path: frontend/dist
```

4. Backend Job (Test and Build)

```yaml
test-backend:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: 20
    - name: Install dependencies
      run: npm ci --prefix backend
    - name: Run tests
      run: npm test --prefix backend
    - name: Build backend
      run: npm run build --prefix backend
    - name: Upload backend artifact
      uses: actions/upload-artifact@v4
      with:
        name: backend-build
        path: backend/dist
```

5. Authenticate to AWS with OIDC

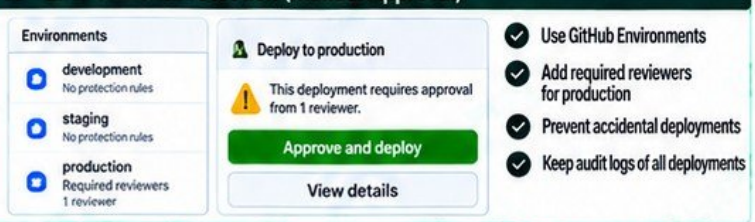
```yaml
- name: Configure AWS credentials (OIDC)
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
    aws-region: us-east-1
```

- Use OpenID Connect (OIDC) for secure authentication
- No long-lived AWS keys in GitHub
- Configure IAM role with trust relationship for GitHub
- Use least privilege permissions

6. Deploy Job (After Approval)

```yaml
deploy:
  needs: [test-frontend, test-backend]
  runs-on: ubuntu-latest
  environment:
    name: production
    url: https://your-domain.com
  steps:
    - name: Download artifacts
      uses: actions/download-artifact@v4
    - name: Deploy frontend to S3
      run: aws s3 sync frontend/dist s3://taskflow-frontend --delete
    - name: Invalidate CloudFront
      run: aws cloudfront create-invalidation --distribution-id E123ABC --paths "/*"
    - name: Deploy backend (EC2)
      run: |
        ssh ec2-user@${{ secrets.EC2_HOST }} << 'EOF'
        cd /opt/taskflow
        sudo systemctl restart taskflow-api
```

7. Environment Protection (Manual Approval)



- Use GitHub Environments
- Add required reviewers for production
- Prevent accidental deployments
- Keep audit logs of all deployments

9. Common Issues and Fixes

Issue	Cause	Fix
OIDC authentication failed	Incorrect IAM trust policy	Update trust policy for GitHub OIDC
Tests failing	Missing dependencies	Check package.json and install steps
Build artifact not found	Wrong artifact path	Verify upload/download paths
Deployment permission error	Insufficient IAM permissions	Update IAM role with required policies
Manual approval not triggered	Environment not configured	Add required reviewers to environment
CloudFront not updated	Missing invalidation step	Add CloudFront invalidation command
EC2 deploy fails	SSH or service issue	Check EC2 access and systemd logs

9. Security Best Practices

- Use OIDC instead of long-lived AWS keys
- Use least privilege IAM roles
- Keep secrets in GitHub secrets (not in code)
- Scan dependencies (e.g., npm audit)
- Run tests before deployment
- Use environment protection and required reviewers

10. Artifacts Generated

- Frontend build (dist/)
- Backend build (dist/)
- Test reports
- Deployment logs (GitHub Actions)

11. Success Checklist

- Pipeline runs on code push
- Frontend and backend tests pass
- Artifacts are built and uploaded
- AWS authentication works with OIDC
- Deployment happens after approval
- Application is updated and accessible

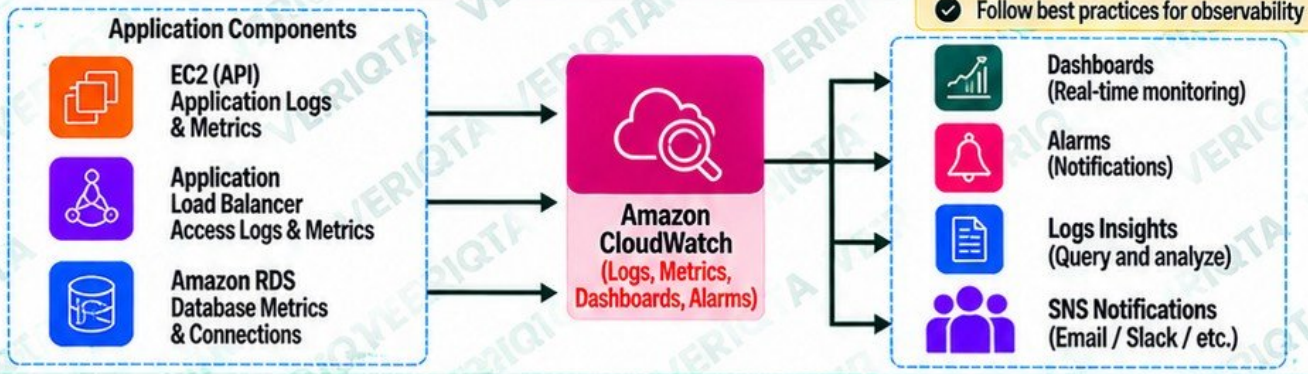
Monitor with CloudWatch

In this step, we collect **API logs and metrics**, **monitor ALB errors**, **EC2 health**, **RDS connections**, and **latency**. We **build a CloudWatch dashboard** and **define actionable alarms**.

Key Objectives

- Collect API logs and application metrics
- Monitor ALB errors and latency
- Monitor EC2 instance health and status
- Monitor RDS connections and performance
- Build a CloudWatch dashboard
- Create actionable alarms and notifications
- Test alerts and investigate issues
- Follow best practices for observability

1. Monitoring Architecture Overview



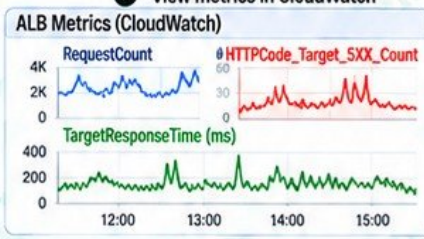
2. Application Logs (EC2 - API)

- Send application logs to CloudWatch
- Log API requests, errors, and responses
- Use structured logs (JSON)
- Use CloudWatch Logs Insights to query

```
# Install and configure CloudWatch agent
sudo yum install -y amazon-cloudwatch-agent
sudo /opt/aws/amazon-cloudwatch-agent/bin/
amazon-cloudwatch-agent-config-wizard
sudo systemctl start amazon-cloudwatch-agent
sudo systemctl enable amazon-cloudwatch-agent
```

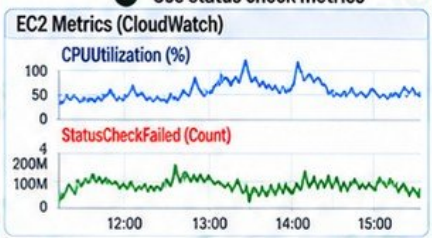
3. ALB Metrics and Logs

- Monitor HTTP 4xx and 5xx errors
- Track request count and latency
- Enable ALB access logs to S3 (optional)
- View metrics in CloudWatch



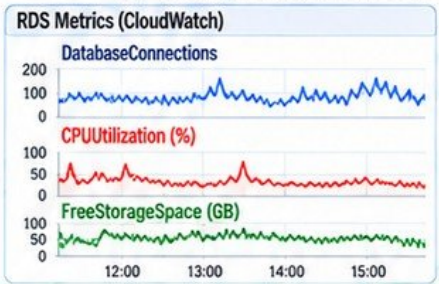
4. EC2 Health Monitoring

- Monitor CPU, memory, disk usage
- Check instance status and health
- Set alarms for high CPU or disk usage
- Use status check metrics



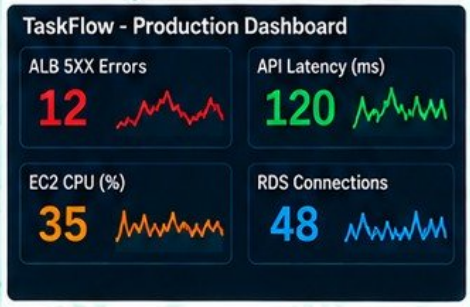
5. RDS Monitoring

- Monitor database connections
- Track CPU, free storage and latency
- Set alarms for high connections or low storage
- Check database health and events



6. Create a CloudWatch Dashboard

- Add key metrics for ALB, EC2, RDS
- Visualize errors, latency and connections
- Use multiple widgets (graphs, numbers)
- Share dashboard with team



7. Create CloudWatch Alarms

- Set alarms for critical metrics
- Send notifications via SNS (email)
- Use meaningful thresholds
- Test alarm by simulating a failure

```
# Example: High CPU alarm
aws cloudwatch put-metric-alarm \
  --alarm-name "High-EC2-CPU"
  --metric-name CPUUtilization \
  --namespace AWS/EC2 \
  --statistic Average
  --period 300
  --threshold 80
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2
  --alarm-actions arn:aws:sns:us-east-1:
    123456789012:veriqta-alerts
```

8. Test and Verify Monitoring

- Generate API traffic and check logs
- Simulate errors (4xx/5xx) and verify alarms
- Stop an EC2 instance to trigger alarm
- Verify SNS notification (email)
- Check dashboard updates in real time

9. Common Issues and Fixes

Issue	Cause	Fix
No application logs	CloudWatch agent not running	Install and start CloudWatch agent
Missing ALB metrics	Wrong region or ALB not selected	Check region and ALB name
No RDS metrics	Enhanced monitoring disabled	Enable Enhanced Monitoring
Alarm not triggering	Threshold too high	Adjust threshold and test
No SNS notification	SNS topic not subscribed	Confirm email subscription
High latency	Backend or DB issue	Check logs and database metrics

10. Best Practices

- Monitor key metrics for all components
- Set actionable and realistic thresholds
- Use structured and centralized logs
- Keep dashboards simple and clear
- Use SNS for critical alerts
- Review alarms and logs regularly
- Set up log retention policies
- Document monitoring and alerting

Troubleshoot Real Failures

In this step, we investigate common real-world failures in the TaskFlow application, including 502 errors, unhealthy targets, database connection timeouts, permission failures, missing environment configuration, and failed deployments. We use logs, metrics, and commands to find the root cause and fix the issues.

Key Objectives

- Diagnose 502 errors and fix the root cause
- Troubleshoot unhealthy ALB targets
- Investigate database connection timeouts
- Resolve permission and IAM failures
- Find missing or incorrect environment variables
- Debug failed deployments (CI/CD)
- Use logs, metrics and tools for evidence
- Apply systematic troubleshooting steps

1. Troubleshooting Workflow

1. **Identify the issue** (error message, symptom)
2. **Gather evidence** (logs, metrics, configurations)
3. **Find the root cause** (compare with expected state)
4. **Apply the fix** (make changes and test)
5. **Verify and monitor** (confirm resolution and prevent recurrence)

2. 502 Bad Gateway Errors

502

- Usually caused by:
- Unhealthy ALB targets
- Backend service not running
- Wrong target group configuration
- Application crashing or timing out
- Network or security group issues

```
# Check ALB target health
aws elbv2 describe-target-health \
 --target-group-arn arn:aws:elasticloadbalancing:...\
 --query 'TargetHealthDescriptions[*].[Target.Id,TargetHealth.State]'

| i-0abc123def    |   healthy     |
| i-0def456ghi    |   unhealthy   |
```

3. Unhealthy Targets

- Check instance health and service status
- Verify application is listening on correct port
- Check security group rules
- Review ALB health check configuration
- Inspect application logs

```
# Check target group health
aws elbv2 describe-target-health \
--target-group-arn arn:aws:elasticloadbalancing:..\

# Check application status on EC2
sudo systemctl status taskflow-api
sudo journalctl -u taskflow-api -n 50
```

4. Database Connection Timeouts

- Check RDS endpoint and port
- Verify security group allows EC2 access
- Confirm database is available
- Check connection pool configuration
- Inspect application logs for timeout errors

```
# Test database connectivity from EC2
telnet taskflow-db.abc123.us-east-1.rds.
amazonaws.com 5432

# Check RDS status
aws rds describe-db-instances \
--query 'DBInstances[0].DBInstanceStatus"
"available"
```

5. Permission Failures

- Check IAM role and attached policies
- Verify required permissions (S3, RDS, etc.)
- Review CloudWatch and AWS error messages
- Ensure correct resource ARNs
- Use least privilege permissions

```
# Example: S3 permission error
An error occurred (AccessDenied) when
calling the PutObject operation:
User: arn:aws:sts::123456789012:assumed-
role/taskflow-role is not authorized

# Check IAM role policies
aws iam list-attached-role-policies \
 --role-name taskflow-role
```

6. Missing Environment Configuration

- Check required environment variables
- Verify .env or parameter store values
- Ensure variables are set in deployment
- Validate application configuration
- Check for typos or missing values

```
# Example: Missing DATABASE_URL
Error: DATABASE_URL is not defined
  at Object.<anonymous> (src/db.js:5:13)

# Check environment variables
printenv | grep -E "DATABASE_URL|PORT|NODE_ENV"
NODE_ENV=production
PORT=3000
```

7. Failed Deployments (CI/CD)

- Check GitHub Actions logs
- Verify build and test steps
- Check AWS authentication (OIDC)
- Ensure artifacts are built correctly
- Validate deployment commands

```
# View GitHub Actions logs
Run npm run build
Error: Build failed with 1 error:
src/App.js:10:5: ERROR: Cannot find
 module 'axios'

# Fix: Install missing dependency
npm install axios
npm run build
```

GitHub Actions

✗ Build	12s
✓ Test	25s
✗ Deploy	5s

8. Useful Debugging Commands

- Check application logs
- Check system service status
- Verify network connectivity
- Inspect environment variables
- Review AWS resource status
- Use browser developer tools

```
# Application logs
sudo journalctl -u taskflow-api -f
# System service status
sudo systemctl status taskflow-api
# Networt test
curl -v http://localhost:3000/api/health
# Check open ports
sudo ss -tuln | grep 3000
```

9. Common Errors and Fixes

Error	Cause	Fix
502 Bad Gateway	Unhealthy targets / app down	Check service, logs, ALB health checks
Target unhealthy	App not running / wrong port	Start service, fix port, check SG rules
Database timeout	Network / SG / DB unavailable	Check RDS status, SG, endpoint, credentials
Access denied	Missing IAM permissions	Attach correct policy to role
Missing env variable	Not set in deployment	Set required environment variables
Build failed	Missing dependency	Install dependency and rebuild
Deploy failed	Invalid AWS credentials	Check OIDC role and permissions
CORS error	Frontend and API domain mismatch	Update CORS configuration
Service not starting	App crash / wrong config	Check logs and configuration

10. Best Practices

- Use logs and metrics for evidence
- Follow a systematic troubleshooting process
- Check recent changes (deployments, configs)
- Verify each component individually
- Document the root cause and solution
- Set up proper monitoring and alarms
- Use meaningful error messages
- Test fixes in a controlled manner
- Implement preventive measures

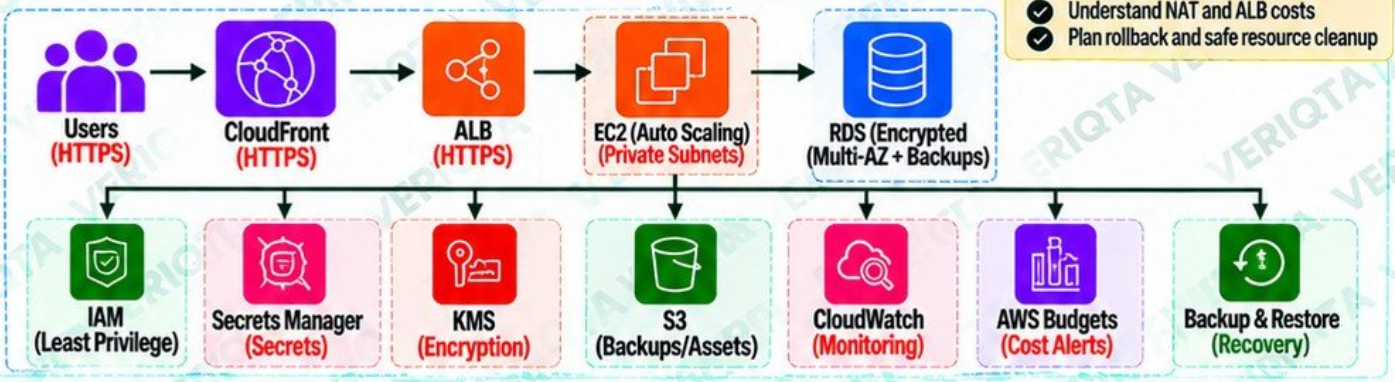
Security, Cost, and Recovery Review

In this step, we review IAM permissions, encryption, private networking, secrets, backups, restore testing, AWS Budgets, NAT and ALB costs, rollback, and safe resource cleanup.

Key Objectives

- Review and apply least privilege IAM
- Ensure encryption in transit and at rest
- Use private networking and security groups
- Manage secrets securely
- Configure backups and test restore
- Set up AWS Budgets and cost alerts
- Understand NAT and ALB costs
- Plan rollback and safe resource cleanup

1. Overall Architecture (Security, Cost and Recovery)



2. IAM Permissions Review

- Use least privilege permissions
- Use IAM roles instead of long-lived keys
- Review policies for unused permissions
- Use condition keys and resource-level access

```
# Review IAM policy for EC2 role
aws iam get-role-policy \
  --role-name taskflow-ec2-role \
  --policy-name TaskFlowPolicy
```

3. Encryption and Data Protection

- Use HTTPS (TLS) for all traffic
- Enable encryption at rest (RDS, S3, EBS)
- Use AWS KMS for key management
- Encrypt secrets and environment variables

```
# Check RDS encryption
aws rds describe-db-instances \
  --db-instance-identifier taskflow-db \
  --query 'DBInstances[0].StorageEncrypted'
# Check S3 bucket encryption
aws s3api get-bucket-encryption \
  --bucket taskflow-frontend
```

4. Private Networking and Security

- Keep EC2 and RDS in private subnets
- Allow only required ports in security groups
- Use VPC endpoints where possible
- Block public database access
- Use NACLs and security group rules

```
# Check security group rules
aws ec2 describe-security-groups \
  --group-ids sg-0abc123def
  --query 'SecurityGroups[0].IpPermissions'
```

5. Secrets Management

- Store secrets in AWS Secrets Manager
- Never hardcode secrets in code
- Use IAM roles to access secrets
- Rotate secrets regularly

```
# Retrieve a secret (example)
aws secretsmanager get-secret-value \
  --secret-id taskflow/db-credentials \
  --query 'SecretString' --output text
```

6. Backups and Restore Testing

- Enable automated RDS backups
- Set appropriate retention period
- Test restoring to a new database
- Keep application data and assets backed up

```
# Restore RDS to a new instance (example)
aws rds restore-db-instance \
  --db-instance-identifier taskflow-db-restore
  --source-db-instance-identifier taskflow-db
  --db-instance-class db.t3.micro
```

7. AWS Budgets and Cost Monitoring

- Create AWS Budgets for monthly costs
- Set alerts for 80% and 100% thresholds
- Monitor NAT Gateway and ALB costs
- Identify and remove unused resources
- Use Cost Explorer to analyze usage

Sample Monthly Costs (us-east-1)

Service	Example Usage	Estimated Cost (USD)
Application Load Balancer	1 ALB (24/7)	$16 – $25
NAT Gateway	1 NAT (24/7)	$32 – $45
EC2 (t3.micro)	2 instances	$15 – $20
RDS (db.t3.micro)	1 instance (Multi-AZ)	$25 – $40
S3 + CloudFront	Frentend assets	$5 – $15

8. Rollback Strategy

- Keep previous application version
- Use blue/green or rolling deployments
- Revert daily infrastructure changes
- Document rollback steps

```
# Rollback to previous container image
aws ecs update-service \
  --cluster taskflow
  --service taskflow-api
  --task-definition taskflow-api:previous
```

9. Safe Resource Cleanup

- Remove unused resources (old AMIs, EBS)
- Delete test environments
- Remove unused Elastic IPs and snapshots
- Review and clean up S3 objects and logs

```
# Delete unused EBS snapshot
aws ec2 delete-snapshot \
  --snapshot-id snap-0abc123def
# Terminate test environment
terraform destroy \
  -var-file="prod/terraform.tfvars"
```

10. Best Practices

- Use least privilege IAM
- Encrypt all data (in transit and at rest)
- Keep resources in private subnets
- Use Secrets Manager for sensitive data
- Configure automated backups and test restore
- Set up AWS Budgets and alerts
- Monitor and optimize NAT and ALB costs
- Have a rollback plan
- Clean up unused resources regularly
- Document all security, cost and recovery settings

11. Common Issues and Fixes

Issue	Cause	Fix
Access denied	Missing IAM permissions	Attach correct policy with least privilege
Cannot connect to RDS	Security group or subnet issue	Allow correct port and check network
High unexpected costs	NAT or ALB usage, unused resources	Review Cost Explorer and remove unused resources
Backup restore fails	Incorrect configuration	Check backup retention and restore parameters
Rollback fails	Previous version not available	Keep previous artifacts and test rollback
Resources not deleted	Dependent resources	Delete in correct order and check dependencies

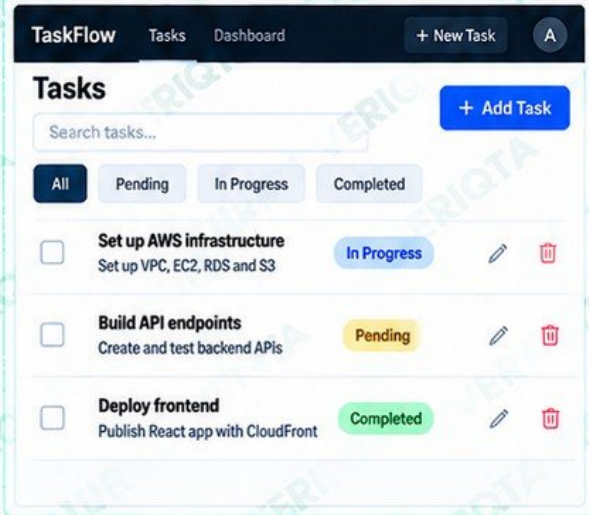
Final Demo and Interview Defense

In this step, we demonstrate the live **TaskFlow** application, explain architecture decisions, show CI/CD and monitoring evidence, simulate a failure, present the GitHub README, and complete an interview-style assessment.

Key Objectives

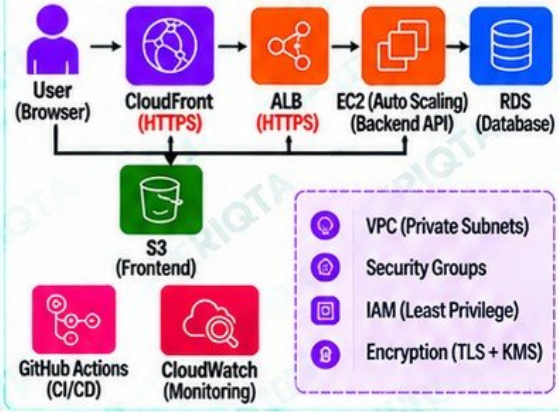
- Demonstrate the live application
- Explain architecture and design choices
- Show CI/CD pipeline evidence
- Present monitoring and alerting
- Simulate a real failure and recover
- Walk through the GitHub README
- Answer interview-style questions
- Show lessons learned and improvements
- Follow best practices and production mindset

1. Live Application Demo



- Open the live application URL
- Create, update, delete and view tasks (CRUD)
- Show responsive design (on mobile and desktop)
- Demonstrate loading indicators
- Show error handling (e.g. invalid input)
- Verify data persistence (in database)

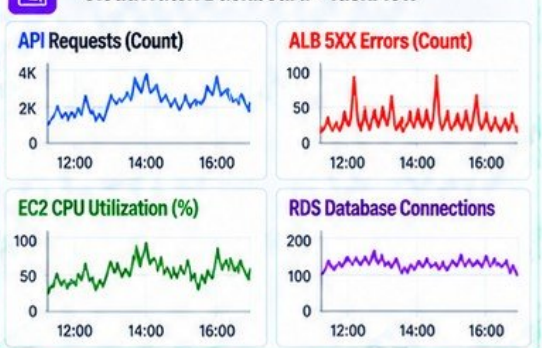
2. Architecture Explanation



3. CI/CD Pipeline Evidence



4. Monitoring Evidence (CloudWatch)

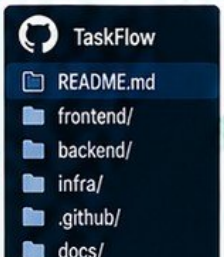


5. Failure Simulation

```
# Simulate backend failure (stop an instance)
aws ec2 stop-instances \
 --instance-ids i-1234567890abcdef0
```

- Verify Auto Scaling replaces unhealthy instance
- Check ALB target health
- Confirm application recovers automatically

6. GitHub README Presentation



README.md (Key Sections)

- Project overview
- Architecture diagram
- Prerequisites
- Environment variables
- Setup instructions
- Deployment steps
- Testing and verification
- Troubleshooting
- Future improvements

7. Interview-Style Assessment

Example Questions

1. Why did you choose this architecture?
2. How does the CI/CD pipeline work?
3. How do you ensure the application is secure?
4. What monitoring and alerting did you set up?
5. How does the application handle failures?
6. How would you reduce the AWS costs?
7. What are the next steps to improve this project?
8. How would you troubleshoot a 502 error?

8. Key Takeaways

- Full-stack application deployed on AWS
- Used infrastructure as code (Terraform)
- Implemented CI/CD with GitHub Actions
- Configured monitoring and alerting
- Demonstrated failure recovery
- Applied security best practices
- Managed costs and cleanup
- Documented everything in GitHub README
- Prepared and answered interview questions

9. Demo Checklist

- Live app is accessible (HTTPS)
- CRUD operations work correctly
- CI/CD pipeline runs successfully
- Monitoring dashboard shows metrics
- Failure simulation and recovery demonstrated
- GitHub README is complete and clear
- All resources are secured and cost-optimized

10. Common Interview Questions and Answers (Examples)

Question	Key Points to Cover
Why this architecture?	Scalability, high availability, cost, security
How does CI/CD work?	GitHub Actions, tests, build, deploy, OIDC
How do you secure it?	IAM, security groups, private subnets, encryption
How do you monitor it?	CloudWatch metrics, logs, alarms, dashboards
How do you handle failures?	Auto Scaling, health checks, retries, rollbacks
How do you reduce costs?	Right sizing, budgets, remove unused resources
How do you troubleshoot 502?	Check ALB targets, logs, instance health, config

11. Next Steps

- Add custom domain
- Implement blue/green deployment
- Add centralized logging (e.g. ELK)
- Integrate notifications (SNS/Slack)
- Add more automated tests
- Improve scalability and resilience
- Continue learning and build more projects